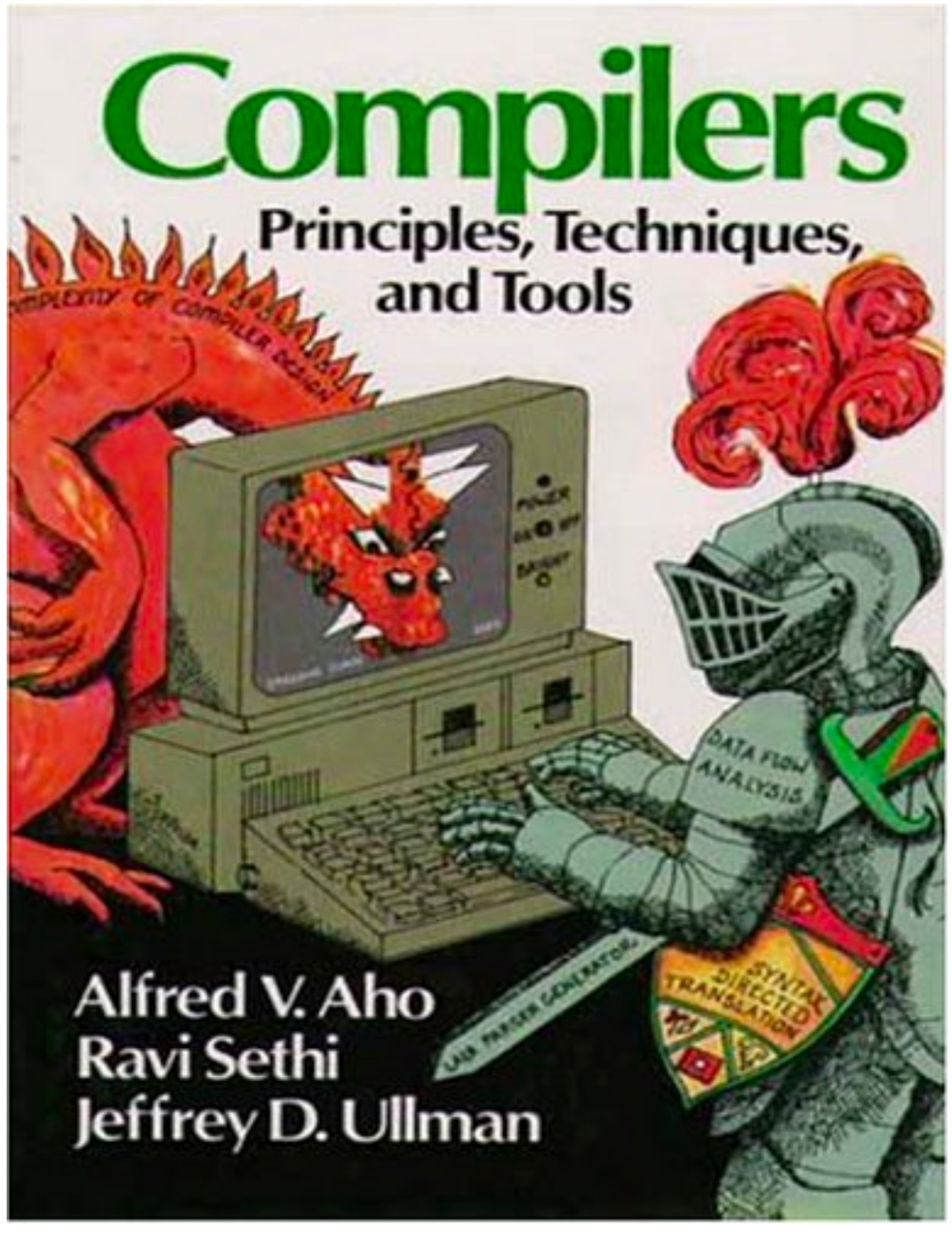
UNIVERSITÄT PASSAU

Software Analysis

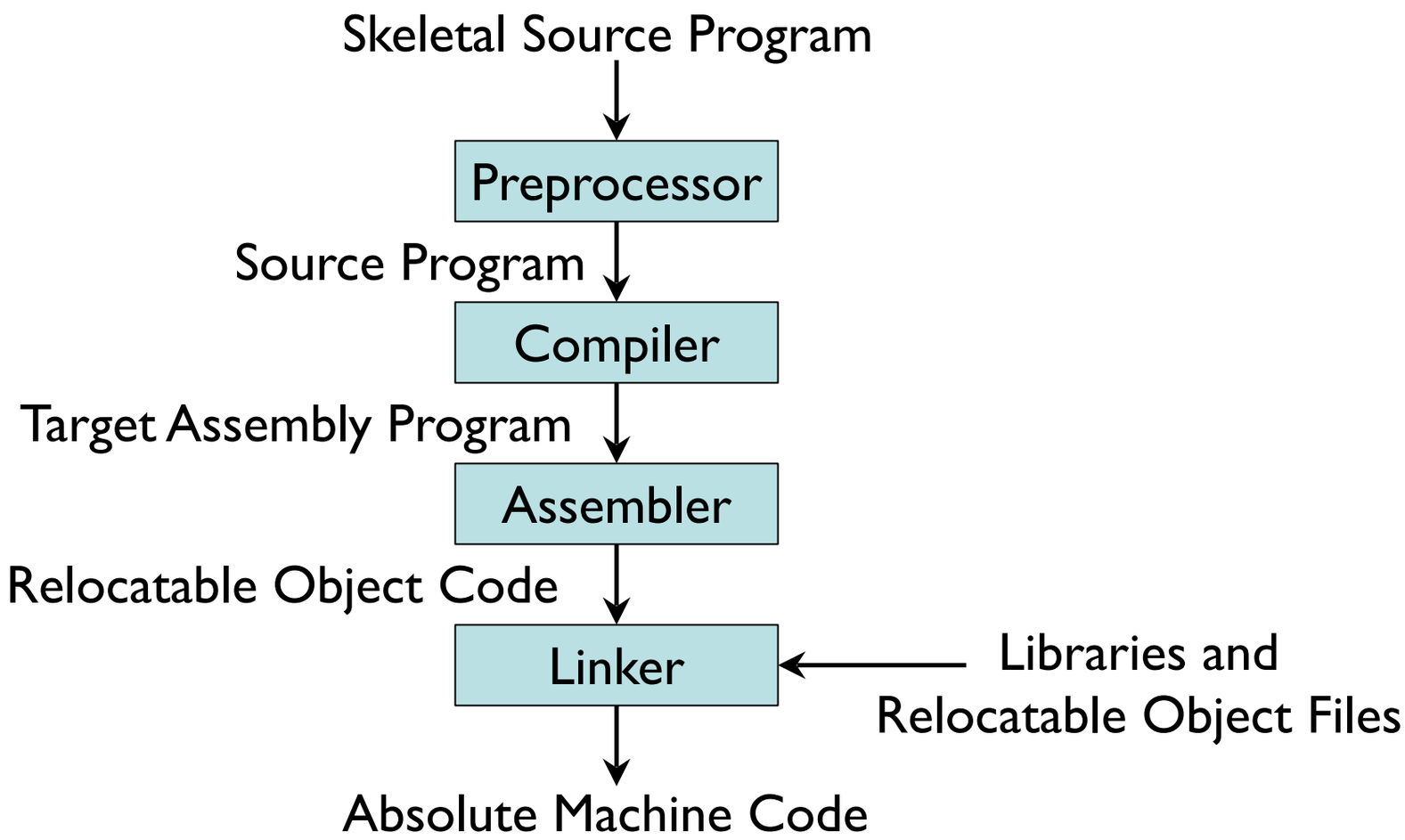
On the Naturalness of Software: Token-Level Analysis

Gordon Fraser
Lehrstuhl für Software Engineering II



Chapters 1 and 2

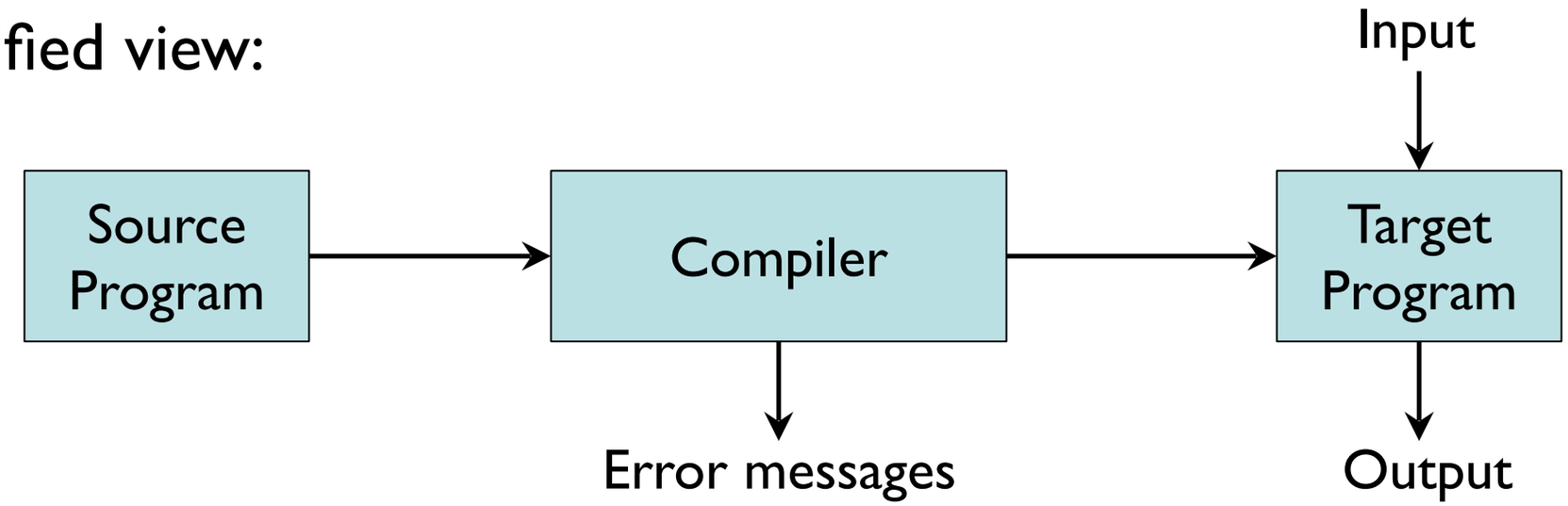
Compiler Toolchain



Compilers

- Compilation
  Translation of a program written in a source language into a semantically equivalent program written in a target language

- Simplified view:

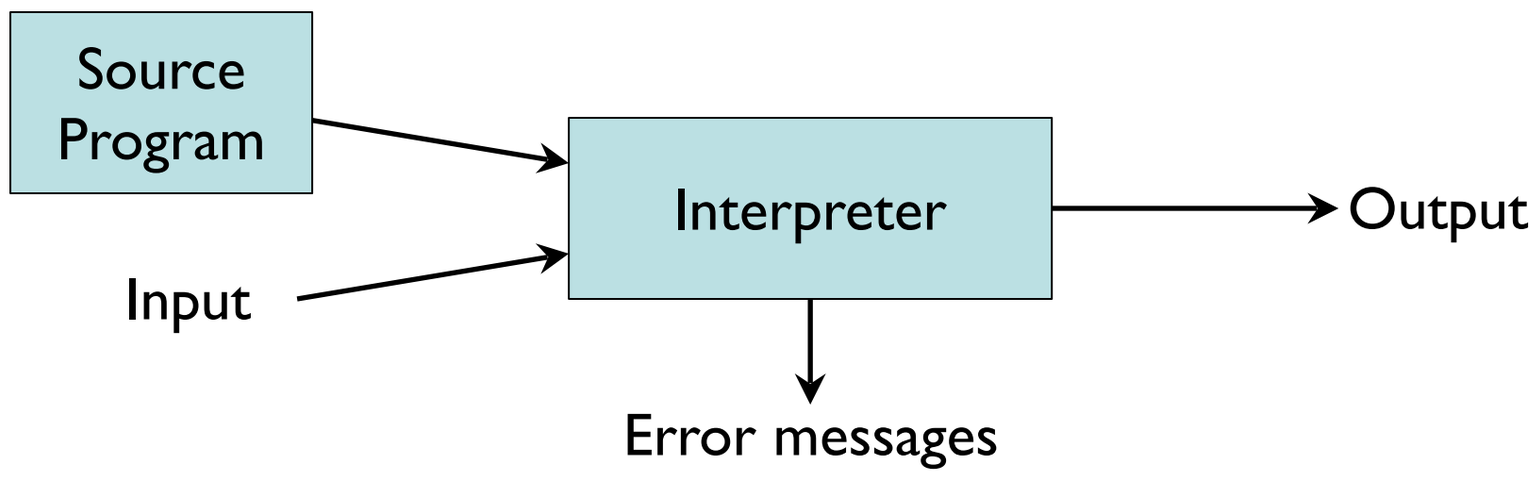


1. *Analysis* determines the operations implied by the source program which are recorded in a tree structure

2. *Synthesis* takes the tree structure and translates the operations therein into the target program

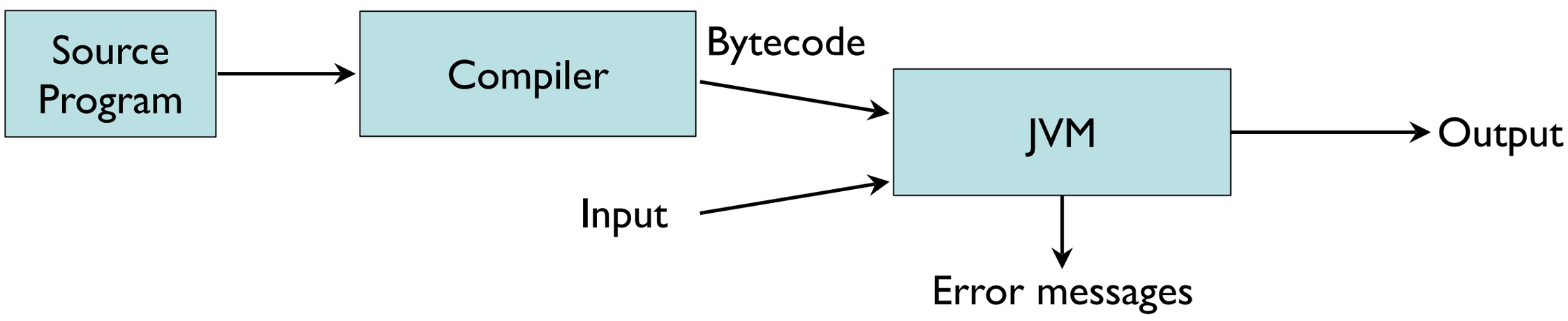
Interpreter

- Interpretation
  Performing the operations implied by the source program
- Simplified view:



Interpreter

- Interpretation
  Performing the operations implied by the source program
- Simplified view:



Syntax Analysis

- The syntax analysis portion of a language processor nearly always consists of two parts:
  - A low-level part called a ***lexical analyser*** (mathematically, a finite automaton based on a regular grammar)
  - A high-level part called a ***syntax analyser***, or parser (mathematically, a push-down automaton based on a context-free grammar, or BNF)

- *Simplicity* - less complex approaches can be used for lexical analysis; separating them simplifies the parser

- *Efficiency* - separation allows optimisation of the lexical analyser

- *Portability* - parts of the lexical analyser may not be portable, but the parser always is portable

Lexical Analysis

- A lexical analyser is a pattern matcher for character strings
- Identifies substrings of the source program that belong together
- *Lexemes* match a character pattern, which is associated with a lexical category called a *token*
- A *token* is the name for a set of lexemes, all of which have the same grammatical significance for the parser

```
result = oldsum  -  value  /  100;
```

Token	Lexeme
IDENT	result
ASSIGN-OP	=
IDENT	oldsum
SUBTRACT-OP	-
IDENT	value
DIVISION-OP	/
INT-LIT	100
SEMICOLON	;

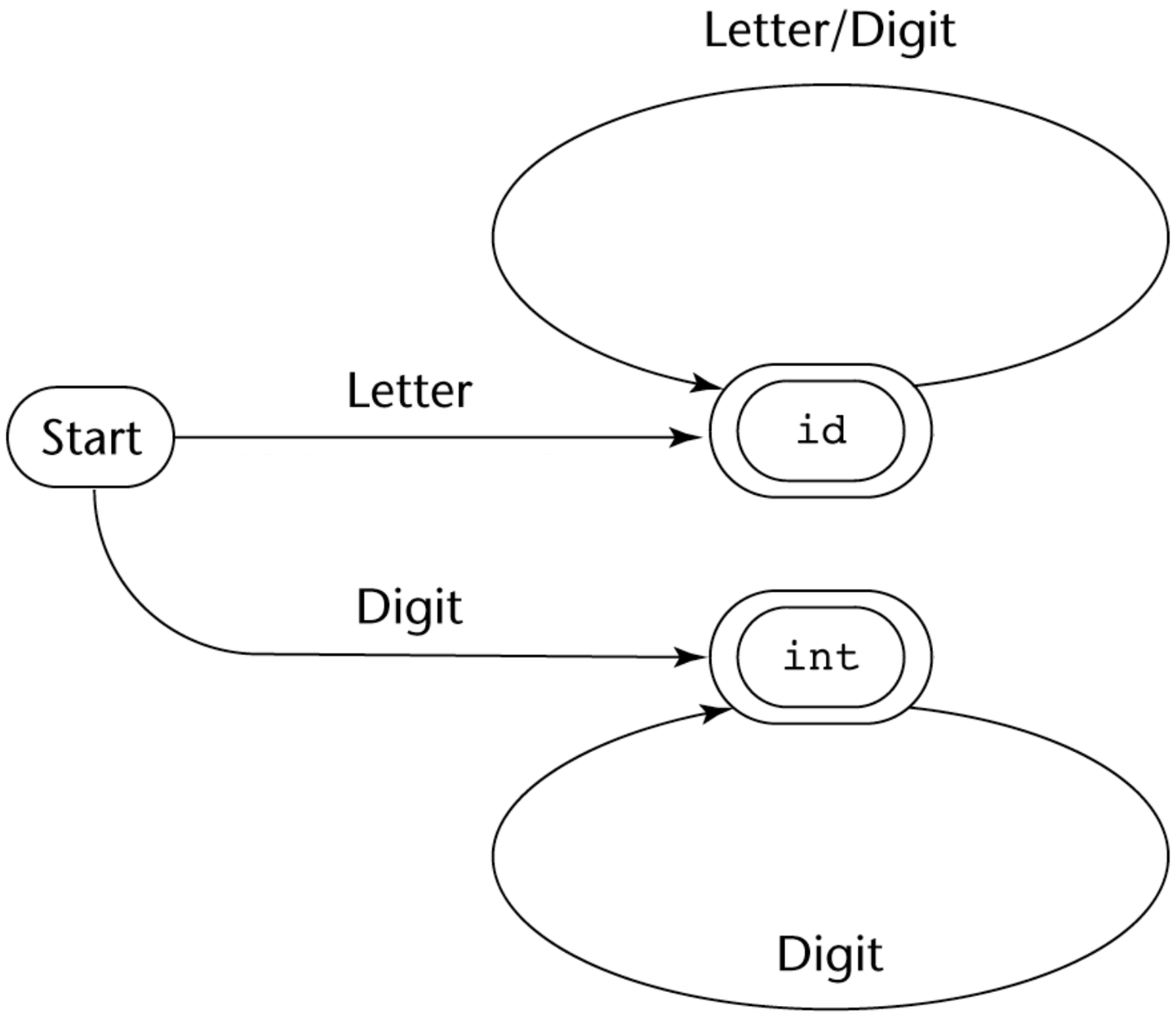
Lexical Analysis

- The lexical analyser is usually a function that is called by the parser when it needs the next token.
- It skips comments and white spaces.
- It inserts lexemes into the ***symbol table*** for later processing by the compiler.
- It detects syntactic errors in tokens, such as ill-formed floating point literals, and reports such errors to the user.
- Specified using either:
  - A formal description of the tokens in regular expressions; a software tool (such as lex) then generates a lexical analyser.
  - A state transition diagram that describes the tokens; this is implemented as a program.

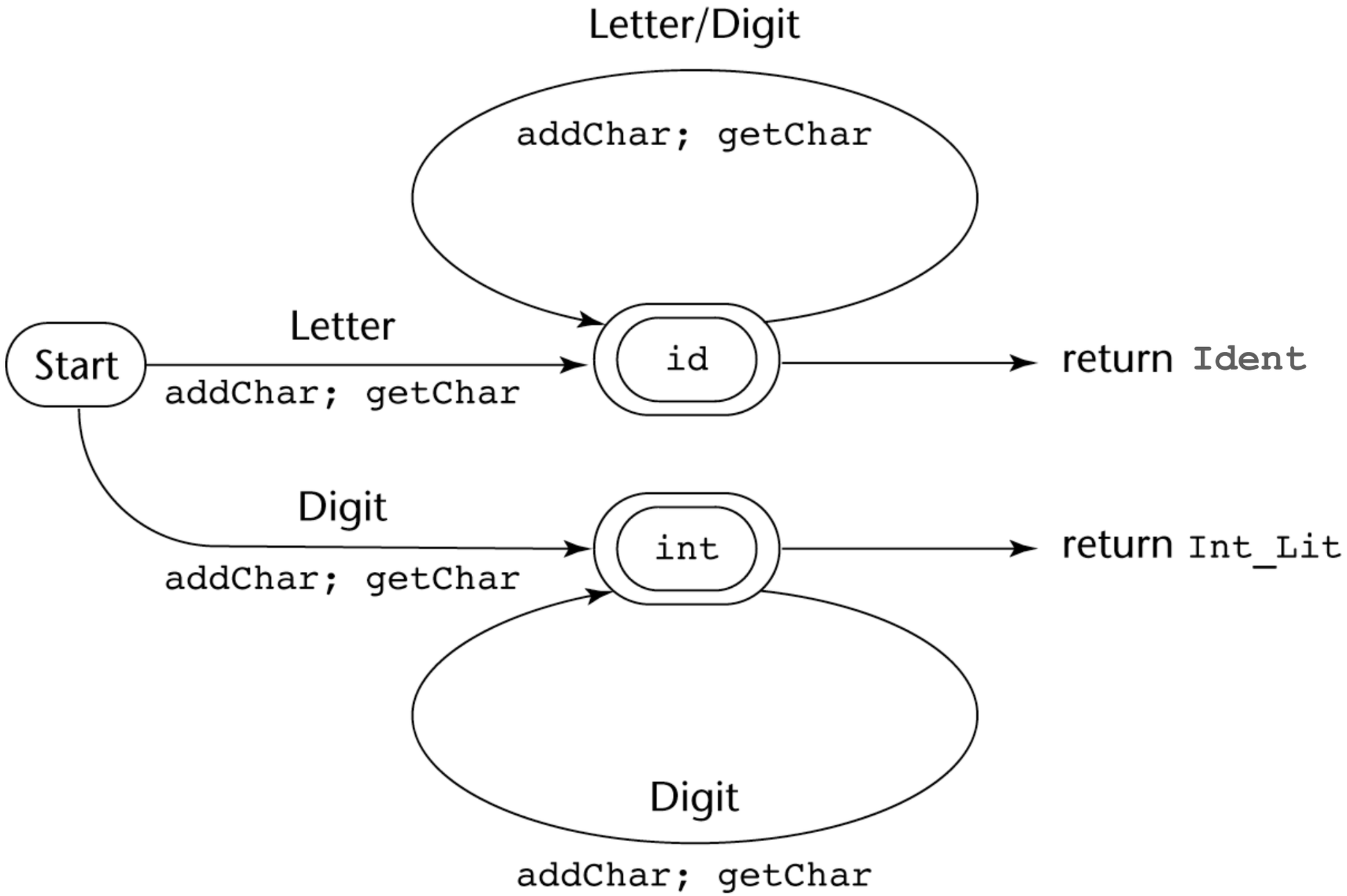
Example

- Assume we need a lexical analyser that recognises:
  - Integers
    - One or more digits.
  - Names
    - Strings of uppercase letters, lowercase letters and digits.
    - It must begin with a letter.
    - Has no length limitation.

State Diagram



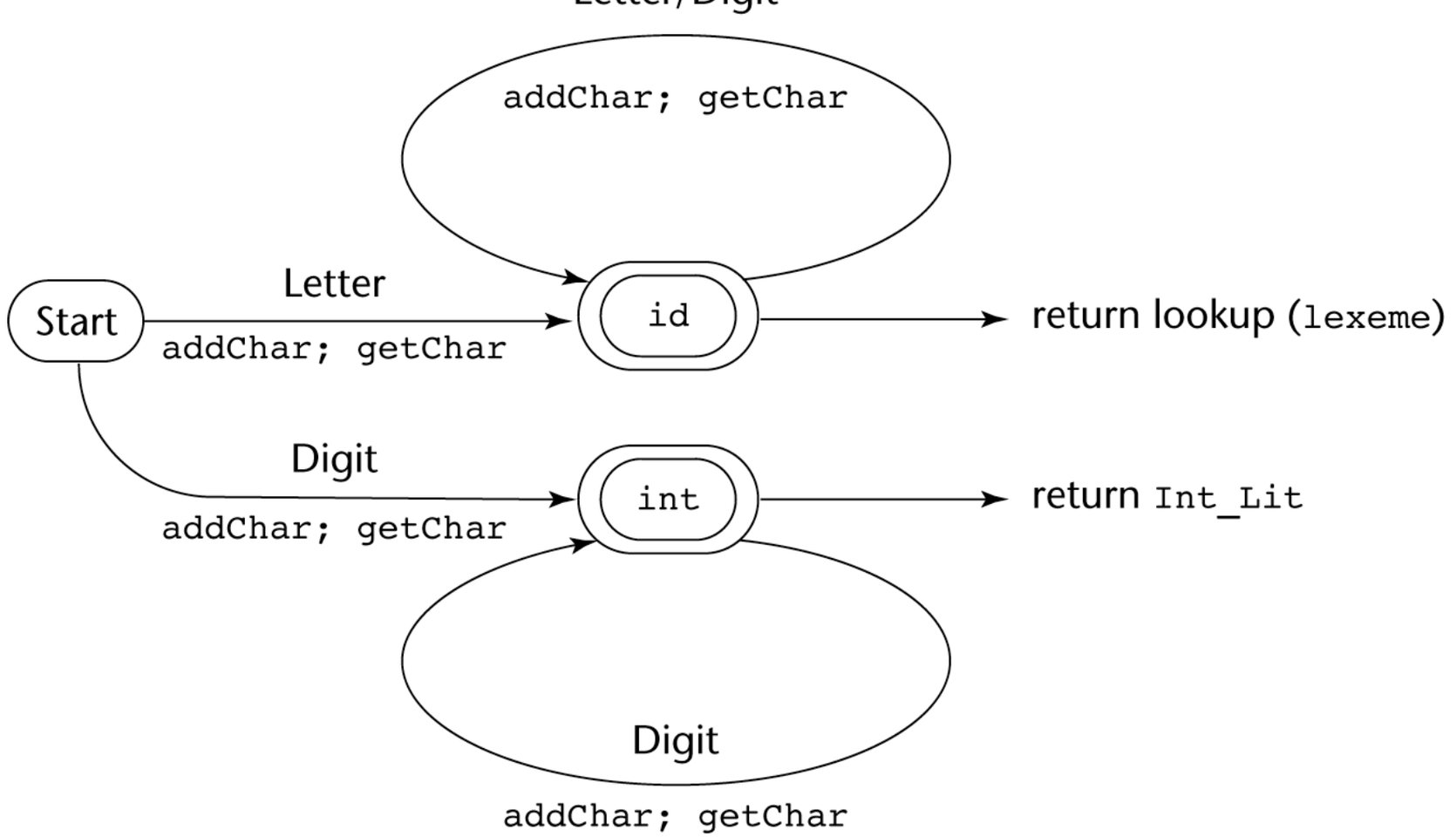
State Diagram



State Diagram

— Reserved words

- It is much simpler and faster to have lexical analyser recognise names and reserved words with the same pattern and use a lookup in a table of reserved words to determine which names are reserved words.



Implementation of the State Diagram

```
/* lex - a simple lexical analyzer*/
int lex() {
    getChar();
    switch(charclass) {

     /* Parse identifiers and reserved words */
  case LETTER:
    addChar();
    getChar();
    while (charClass == LETTER || charClass == DIGIT) {
      addChar();
      getChar();
    }
    return lookup(lexeme);
    break;
  …
```

Lexical Analysis (cont.)

```
…
/* Parse integer literals */
    case DIGIT:
      addChar();
      getChar();
      while (charClass == DIGIT) {
        addChar();
        getChar();
      }
      return INT_LIT;
      break;
  }  /* End of switch */
}  /* End of function lex */
```

Lex/Flex

```
%{
#include <stdio.h>
%}

%option noyywrap

%%
[0-9]+  {
            printf("Saw an integer: %s\n", yytext);
        }

[A-Za-z][A-Za-z0-9]+  {
            printf("Saw an id: %s\n", yytext);
        }

.|\n    {   /* Ignore all other characters. */   }

%%

/*** C Code section ***/
int main(void)
{
    /* Call the lexer, then quit. */
    yylex();
    return 0;
}
```

CCFinder

```
int main() {
  int i = 0;
  static int j = 5;
  while(i < 20) {
    i = i + j;
  }
  std::cout << “Hello World” << i << std::endl;
  return 0;
}
```

- Step 1: Convert a program with multiple files to a single long token sequence

- Step 2: Find longest common subsequence of tokens

```
int main() {
    int i = 0;
    static int j = 5;
    while(i < 20) {
        i = i + j;
    }
    std::cout << “Hello World” << i << std::endl;
    return 0;
}
```

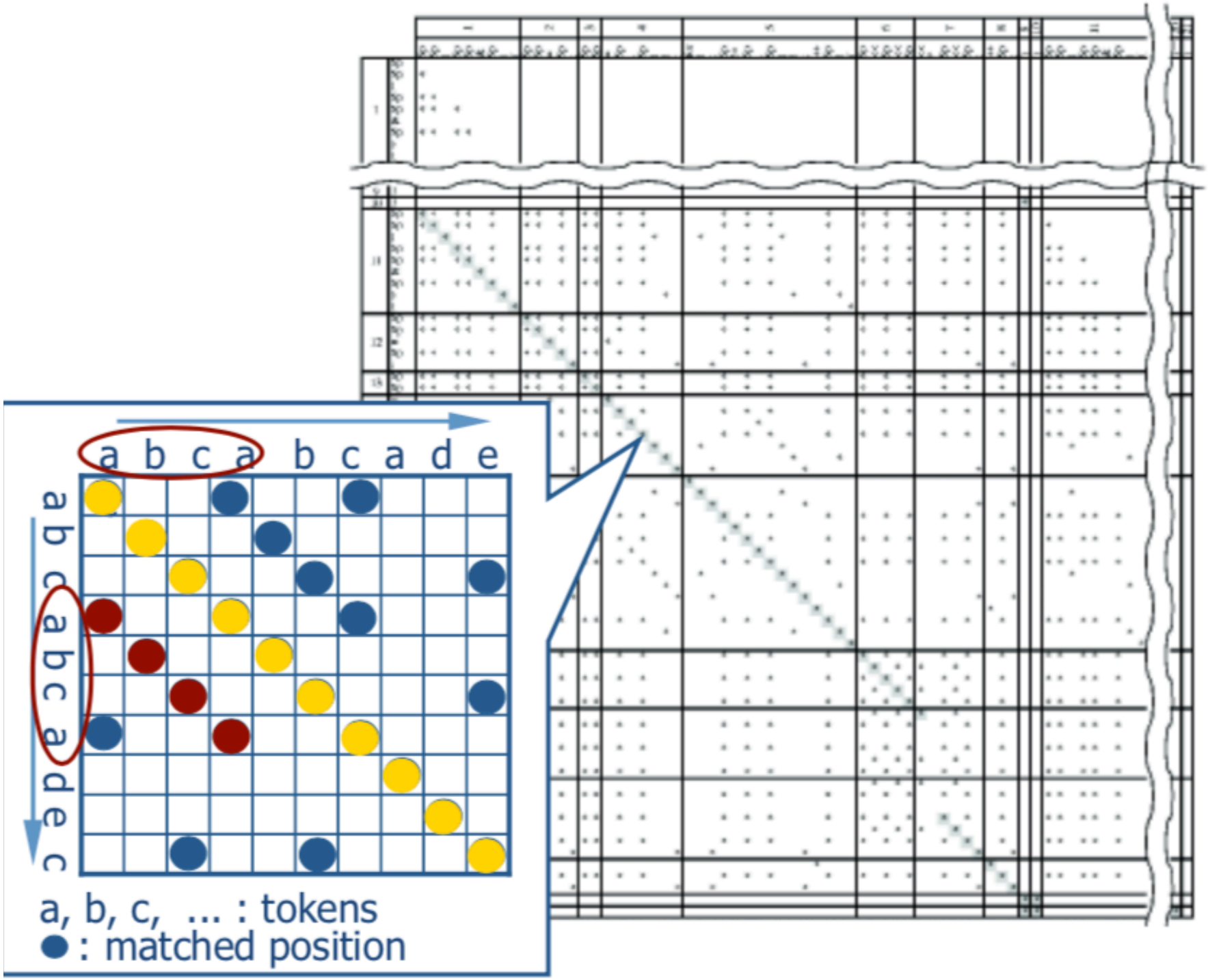
```
int main() {
int i = 0;
static int j = 5;
while(i < 20) {
i = i + j;
}
std::cout << “Hello World” << i << std::endl;
return 0;
}
```

```
int main() {
int i = 0;
static int j = 5;
while(i < 20) {
i = i + j;
}
std::cout << “Hello World” << i << std::endl;
return 0;
}
```

```
int main() {
int i = 0;
int j = 5;
while(i < 20) {
i = i + j;
}
cout << “Hello World” << i << endl;
return 0;
}
```

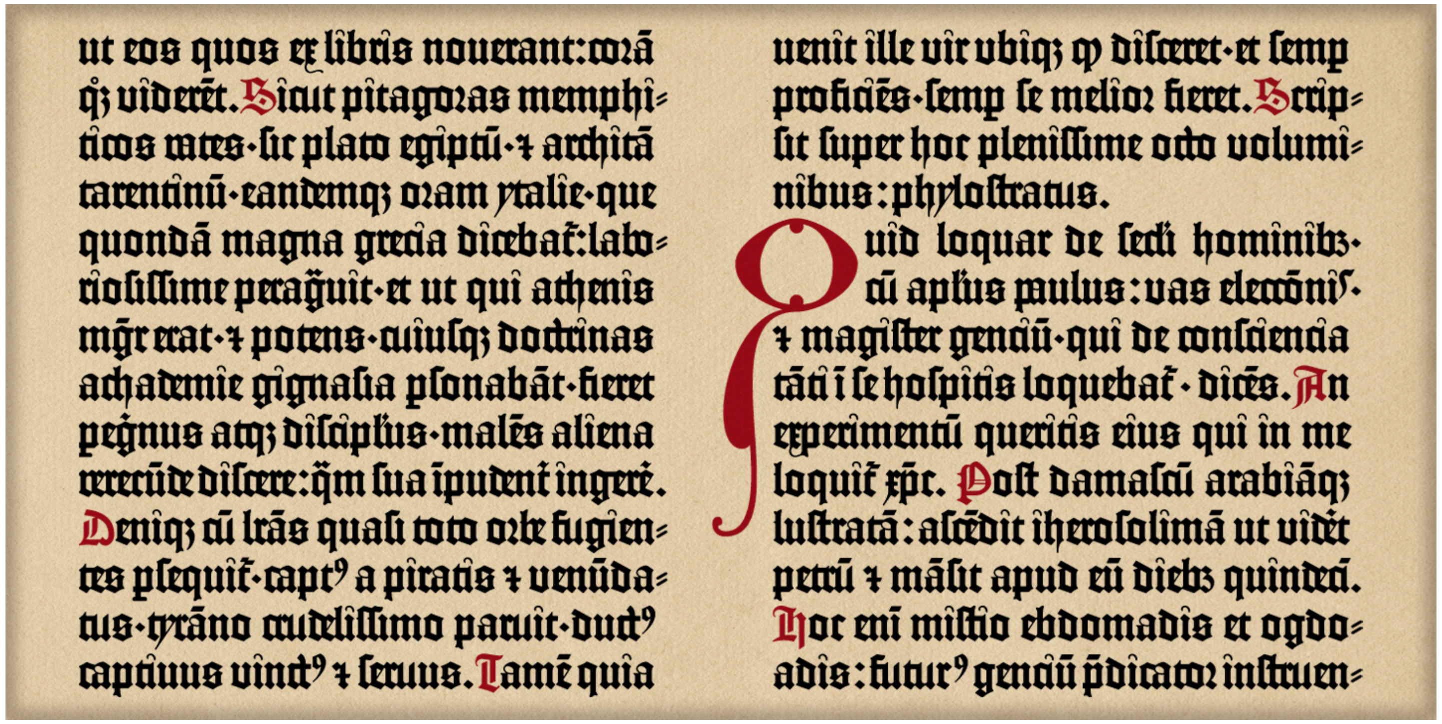
```
int main() {
int i = 0;
int j = 5;
while(i < 20) {
i = i + j;
}
cout << “Hello World” << i << endl;
return 0;
}
```

```
$p $p() {
$p $p = $p;
$p $p = $p;
while($p < $p) {
$p = $p + $p;
}
$p << $p << $p << $p;
return $p;
}
```


a b c a b c a d e
a b c a d e c
a, b, c, ... : tokens
● : matched position

Naturalness of Language

- Speech and writing are **natural** products of human effort
- Natural language processing (NLP) relies on large text bodies to estimate statistical models and regularities in the text
- The objective is to assign a probability to a sentence


ut eos quos ex libris noverant: corã
q̃z viderẽt. Sicut pitagoras memphi-
ticos vates - sic plato egiptũ - et architã
tarentinũ - eandemq; oram ytalie - que
quondã magna grecia dicebat: labo-
riosissime peragravit - et ut qui athenis
mgr erat - et potens - cuiusq; doctrinas
achademie gignasia plonabant - fieret
pegrinus atq; discipulus - malens aliena
verecũde discere: q̃m sua ipudent ingerere.
Deniq; cũ lrãs quasi toto orbe fugien-
tes psequit - capt9 a piratis et venũda-
tus - tyranno crudelissimo paruit - duct9
captivus vinct9 et servus. Tamẽ quia
venit ille vir ubiq; q discerẽt - et semp
pficiẽs - semp se melior fieret. Scrip-
sit super hoc plenissime octo volumi-
nibus: phylostratus.
Quid loquar de seculi hominibz -
cũ apls paulus: vas electõis -
et magister gentiũ - qui de consciencia
tanti i se hospitis loquebat - dicẽs. An
experimentũ queritis eius qui in me
loquit xpc. Post damascũ arabiãq;
lustratã: ascẽdit ierosolimã ut vidẽt
petrũ et mãsit apud eũ diebz quindecĩ.
Hoc eni mistio ebdomadis et ogdo-
adis: futur9 gentiũ pdicator instruen-

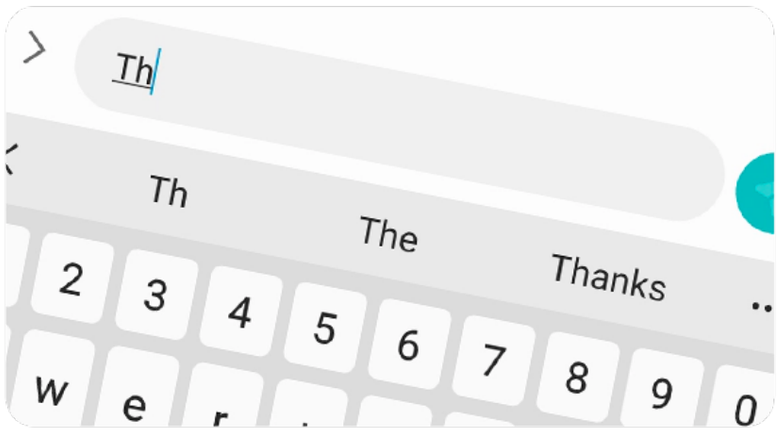
Naturalness of Language

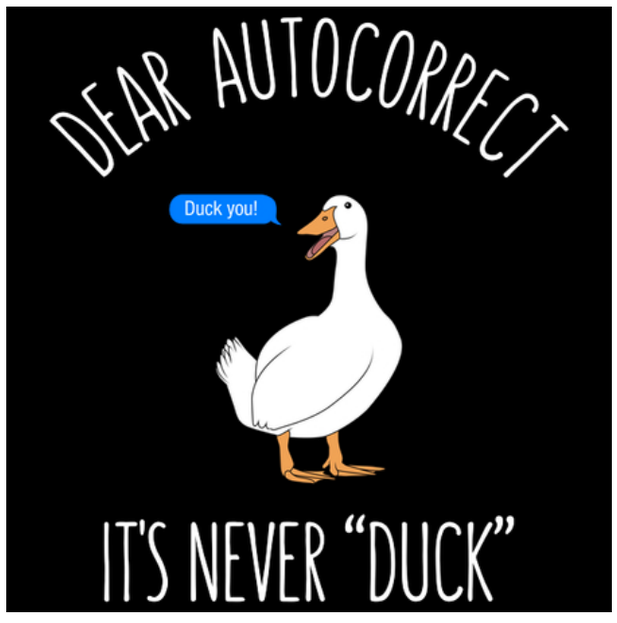
- Machine translation:
  P(high winds tonight) > P(large winds tonight)

- Spell correction:
  P(The lecture lasts ninety minutes) > P(The lecture lasts ninety minuets)

- Speech recognition:
  P(I saw a van) > P(eyes awe of an)

- Others: summarization, question-answering, etc.





Naturalness of Software

- Programming is a form of communication
  - Source code is not written for computers, but for other humans to read
  - Source code written by people has similar regularities as texts in natural languages

- **Can we apply NLP to source code (and SE Tasks)?**

Language Models

- A language model assigns a probability to **utterances** (sequences of words), which are a program in our case
- Documents can be seen as **sequences of tokens**

  $a_1 a_2 \ldots a_i \ldots a_n$

- The probability of a document $p(s)$ is the probability to observe its tokens one after the other
- $p(s)$ can be estimated using the product of a series of conditional probabilities

  $p(s) = p(a_1)p(a_2|a_1)p(a_3|a_1 a_2)\ldots p(a_n|a_1 \ldots a_{n-1})$

Markov Property

- "Only the **past** matters"
- Token occurrences are influenced only by the n-1 tokens that precede the token under consideration
- Example: with n = 4:

  $p(a_i|a_1 \ldots a_{i-1}) \simeq p(a_i|a_{i-3}\, a_{i-2}\, a_{i-1})$

  $p(s) \simeq \prod_i p(a_i|\, a_{i-3} \ldots a_{i-1})$

Unigram Model (n=1)

- $P(a_1 \ldots a_n) \simeq \prod_i P(a_i)$
- There's no conditional probability

- Some automatically generated sentences from a unigram model:
  - fifth, an, of, futures, the, an, incorporated, a, a, the, inflation, most, dollars, quarter, in, is, mass
  - thrift, did, eighty, said, hard, july, bullish
  - that, or, limited, the

Bigram Model (n=2)

- $P(a_1 \ldots a_n) \simeq \prod_i P(a_i|a_{i-1})$
- Probability conditioned on the **previous** word:
  $p(a_i|a_1 \ldots a_{i-1}) \simeq p(a_i|a_{i-1})$
- Some automatically generated sentences from a bigram model:
  - texaco, rose, one, in, this, issue, is, pursuing, growth, in, a, boiler, house, said, mr, gurria, mexico, motion, control, proposal, without, permission, from, five, hundred, fifty, five, yen
  - outside, new, car, parking, lot, of, the, agreement, reached
  - this, would, be, a, record, november

n-Gram Models

- Models are **estimated** on a corpus using **maximum-likelihood** based **frequency-counting** of token sequences

$$p(a_i \mid a_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

- Assuming * represents any token, we estimate the probability that a_4 follows a_1, a_2, a_3:

$$p(a_4 \mid a_1 a_2 a_3) = \frac{count(a_1 a_2 a_3 a_4)}{count(a_1 a_2 a_3 *)}$$

Maximum Likelihood Example

```
<s>I am Sam</s>
<s>Sam I am</s>
<s>I do not like green eggs and ham</s>
```

$$p(a_i \mid a_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

Maximum Likelihood Example

<s>I am Sam</s>
<s>Sam I am</s>
<s>I do not like green eggs and ham</s>

P(I | <s>) = 2/3 = 0.67

$$p(a_i \mid a_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

Maximum Likelihood Example

<s>I am Sam</s>
<s>Sam I am</s>
<s>I do not like green eggs and ham</s>

P(I | <s>) = 2/3 = 0.67

P(</s> | Sam) = 1/2 = 0.5

$$p(a_i \mid a_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

Maximum-Likelihood Example

```
<s>I am Sam</s>
<s>Sam I am</s>
<s>I do not like green eggs and ham</s>
```

P(I | <s>) = 2/3 = 067

P(</s> | Sam) = 1/2 = 05

P(Sam | <s>) = 1/3 = 033

P(Sam | am) = 1/2 = 05

P(am | I) = 2/3 = 067

P(do | I) = 1/3 = 033

$$p(a_i \mid a_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

Example (Corpus)

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what I'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- I'm looking for a good place to eat breakfast
- when is caffe venezia open during the day
- …

Source: Berkeley Restaurant Project (see https://webstanfordedu/~jurafsky/slp3/3pdf)

Example (Raw Bigram Counts)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Out of 9332 sentences

Example (Raw Bigram Prob)

Normalize by unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Result:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Example (Estimation)

I want Chinese food

P(<s>I want chinese food</s>) =

= P(I | <s>) ×
P(want | I) ×
P(chinese | want) ×
P(food | chinese) ×
P(</s> | food)

= .25 × .33 × .0065 × .52 × .68

= .00019

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Computationally easier (and more stable) to use **log probabilities**:

p1 × p2 × p3 = exp(log(p1)+ log(p2)+log(p3))

How Good is my Model?

- Does the language model prefer good sentences to bad ones?
- Assign higher probability to "real" (frequently observed) sentences than "ungrammatical" (rarely observed) ones
- We train parameters of our model on a training set
- We test the model's performance on data we haven't used for training (test set)

Perplexity

- Metric to estimate goodness of models
- The perplexity of a language model on a **test set** is the inverse probability of the test set, normalised by the number of words

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$PP(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_{i-1})}}$$

Perplexity

- Perplexity = weighted average branching factor of a language.
- The branching factor of a language is the number of possible next words that can follow any word.
- Example: Language of digits each of the 10 digits occurs with equal probability P = 1

$$
\begin{aligned}
PP(W) &= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} \\
&= \left(\frac{1}{10}^{N}\right)^{-\frac{1}{N}} \\
&= \frac{1}{10}^{-1} \\
&= 10
\end{aligned}
$$

Perplexity

- Training set: 0 occurs 91 times in the training set, and each of the other digits occurred 1 time each.
- Test set: 0 0 0 0 0 3 0 0 0 0

$$
\begin{aligned}
PP(W) &= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} \\
&= ((\frac{91}{100})^9 \times \frac{1}{100})^{-\frac{1}{10}} \\
&= 1.73
\end{aligned}
$$

Cross-Entropy

- Log-transformed version: **Cross-entropy**:

$$H(s) = -\frac{1}{N} log(P(a_1 \dots a_n))$$

- Cross-entropy measures how surprised a model is by a document

Sparsity

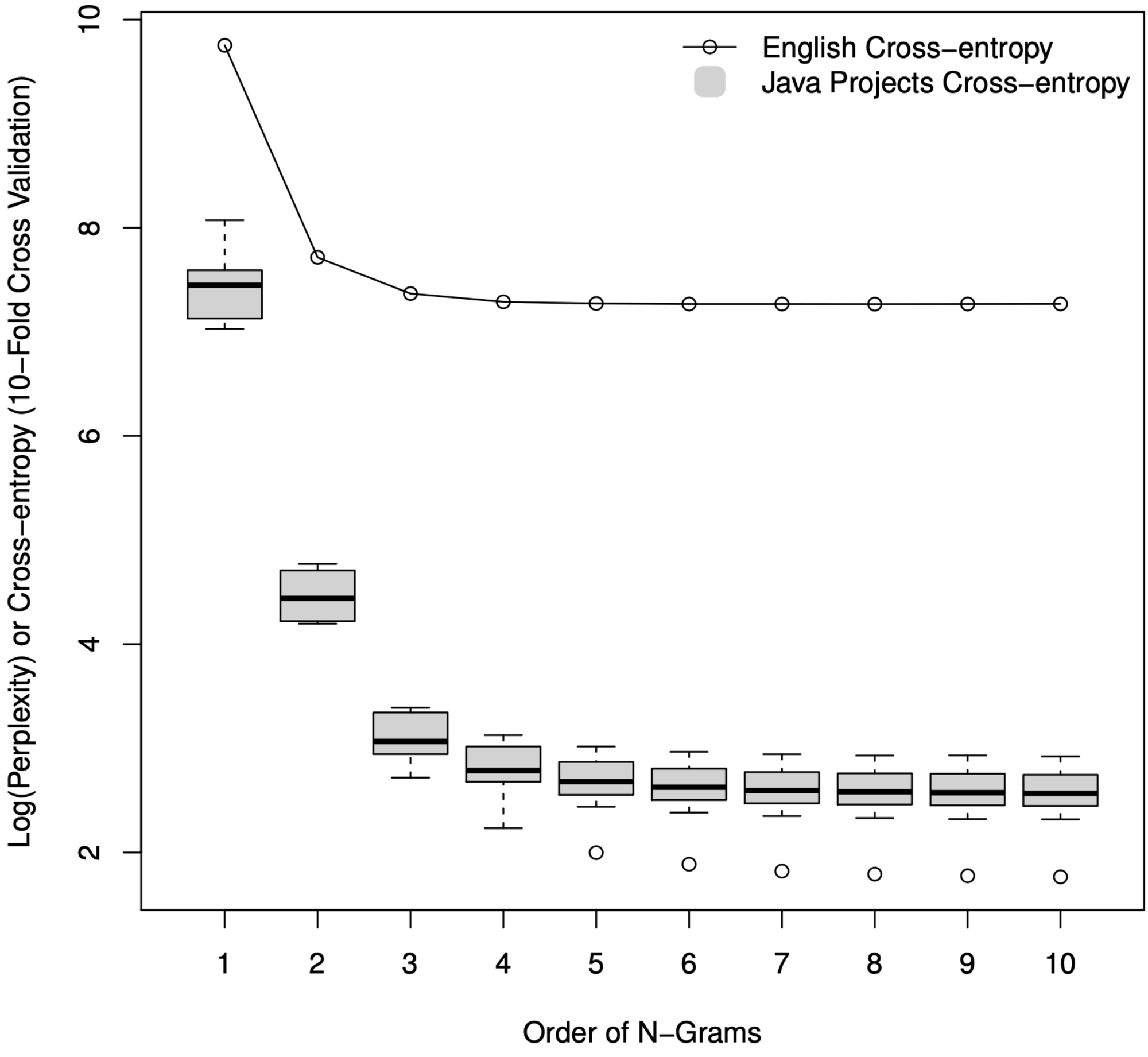
- What if we encounter an **unknown word?**
  - Replace infrequent words with <UNK> to estimate probability
- Given 10^4 token vocabulary a trigram model must estimate 10^{12} coefficients (using a way smaller training corpus)
- What if we encounter a **previously unseen** n-gram (even with known words)?
- Infinitely surprised — probability 0 (infinitely improbably)
- Smoothing techniques try to handle this problem
  - Shave off a bit of probability mass from some more frequent events and give it to the events we've never seen
  - Add-one smoothing (Laplace)
  - Good-Turing estimate, Jelinek-Mercer smoothing (interpolation), Katz smoothing (backoff), Witten-Bell smoothing, Absolute discounting, Kneser-Ney smoothing, Modified Kneser Ney smoothing, …

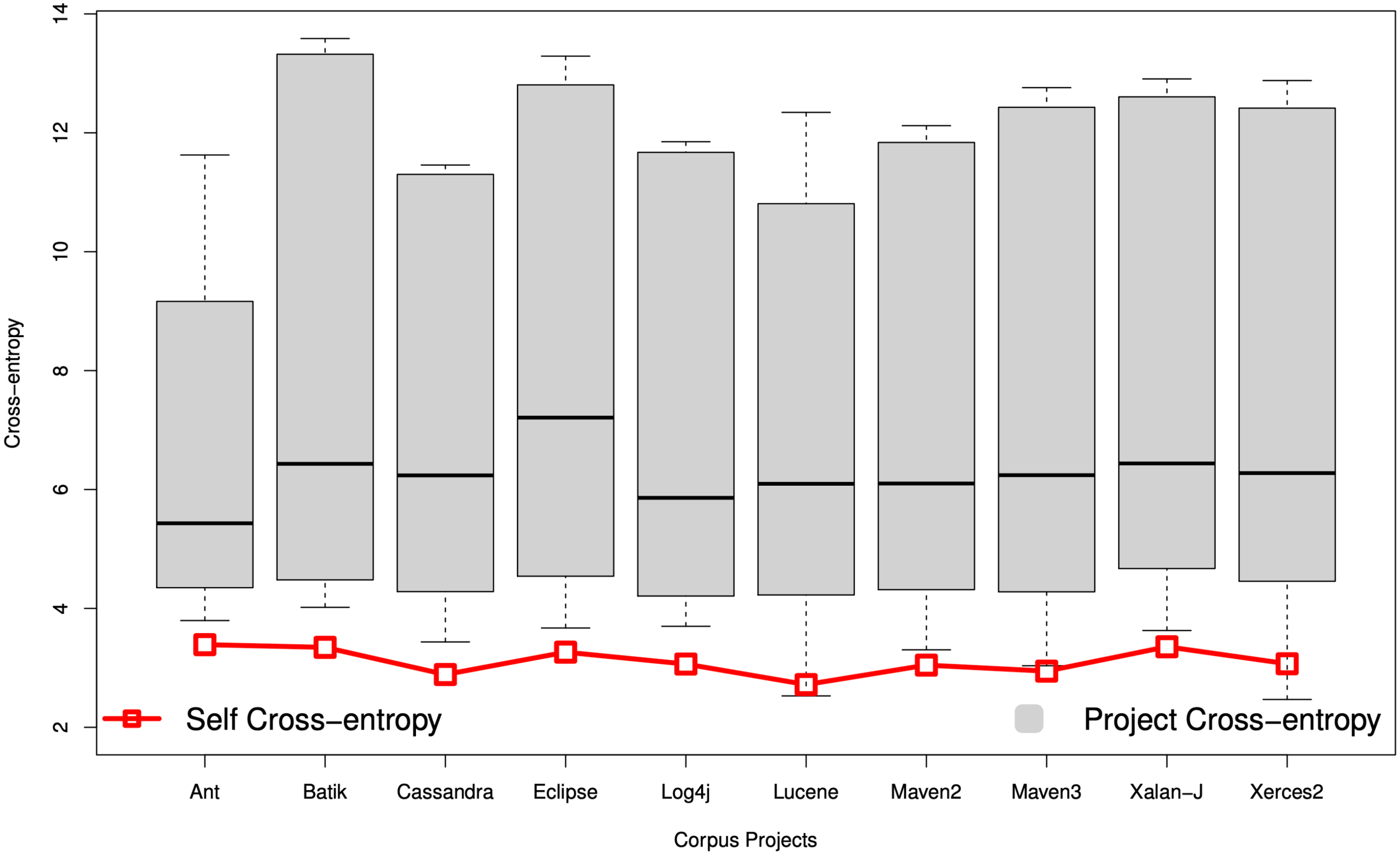
How Natural is Software?

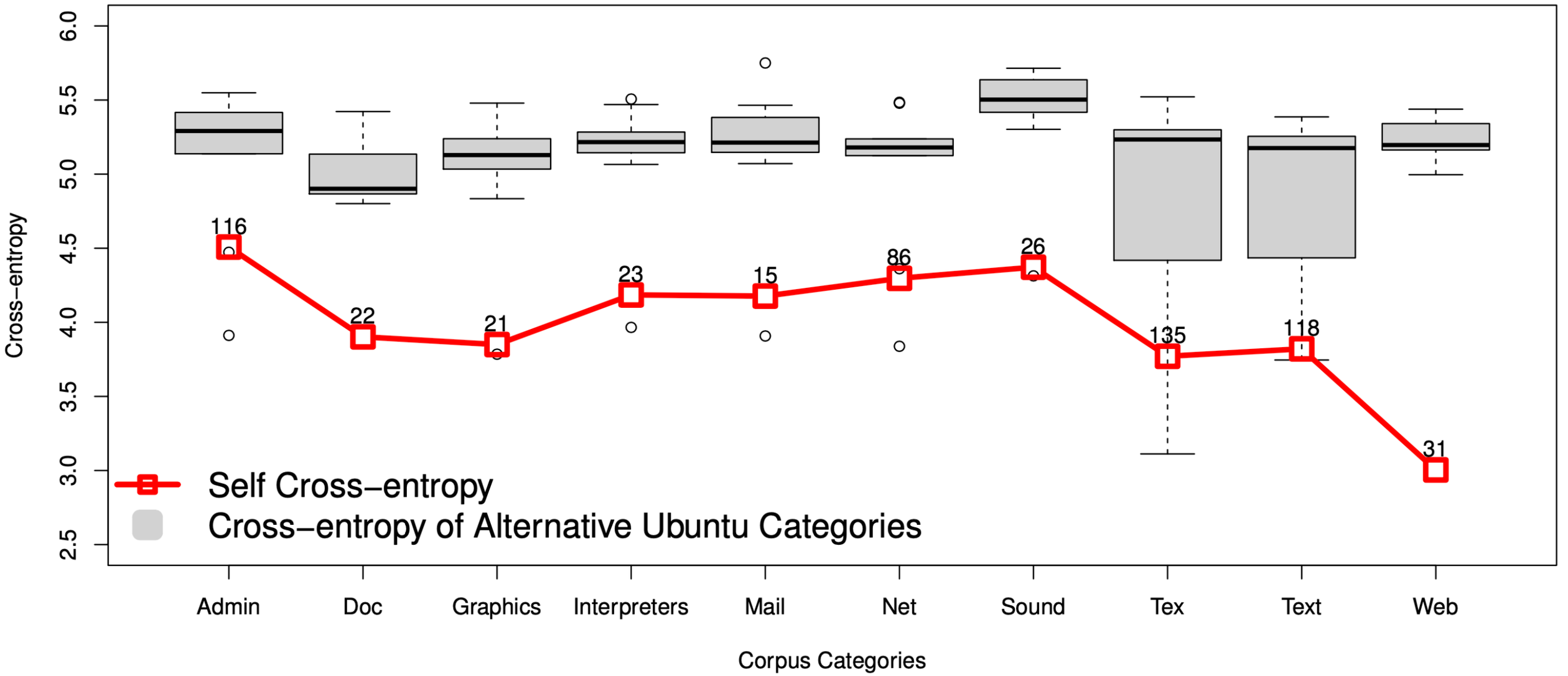
- Hindle, A, Barr, E T, Gabel, M, Su, Z, & Devanbu, P (2016) On the naturalness of software Communications of the ACM, 59(5), 122-131

- Compared cross-entropy on natural language corpus (Gutenberg+Brown) with source code

Java Project	Version	Lines	Tokens	
			Total	Unique
Ant	20110123	254457	919148	27008
Batik	20110118	367293	1384554	30298
Cassandra	20110122	135992	697498	13002
Eclipse-E4	20110426	1543206	6807301	98652
Log4J	20101119	68528	247001	8056
Lucene	20100319	429957	2130349	32676
Maven2	20101118	61622	263831	7637
Maven3	20110122	114527	462397	10839
Xalan-J	20091212	349837	1085022	39383
Xerces	20110111	257572	992623	19542

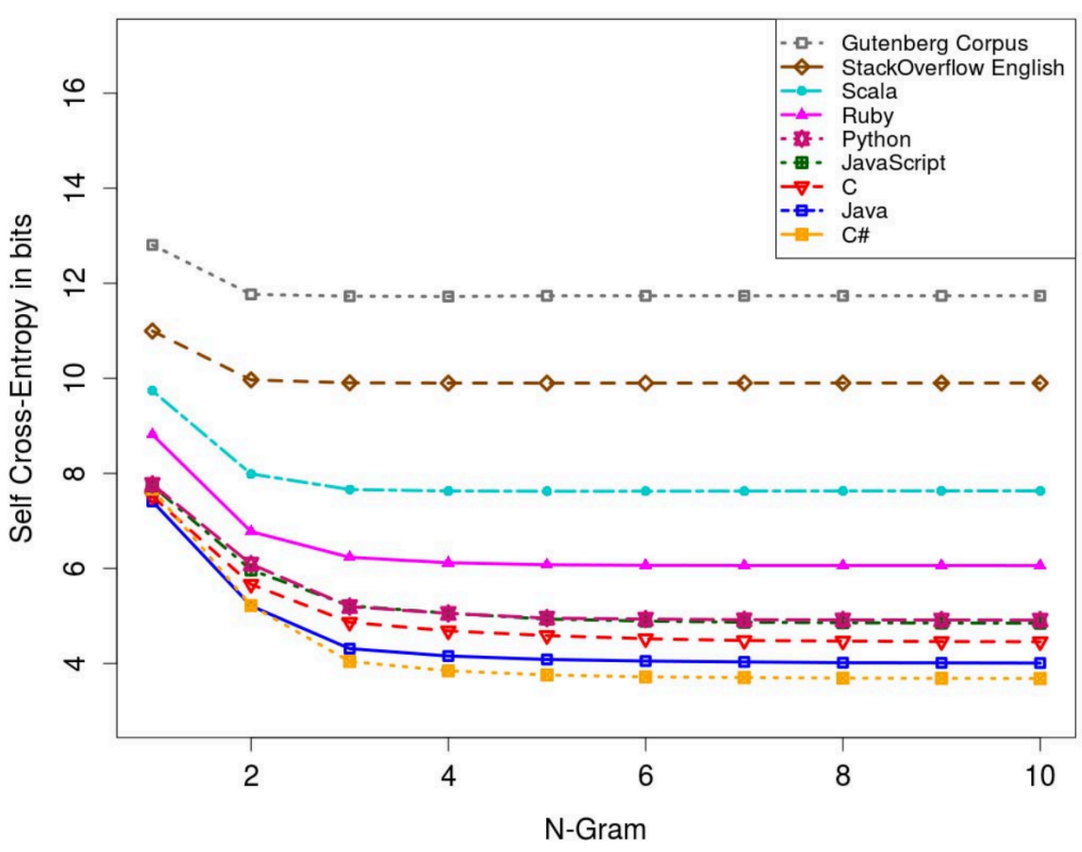

English Cross−entropy
Java Projects Cross−entropy
Log(Perplexity) or Cross−entropy (10−Fold Cross Validation)
2
4
6
8
10
1
2
3
4
5
6
7
8
9
10
Order of N−Grams


Cross−entropy
2
4
6
8
10
12
14
Ant
Batik
Cassandra
Eclipse
Log4j
Lucene
Maven2
Maven3
Xalan−J
Xerces2
Corpus Projects
Self Cross−entropy
Project Cross−entropy

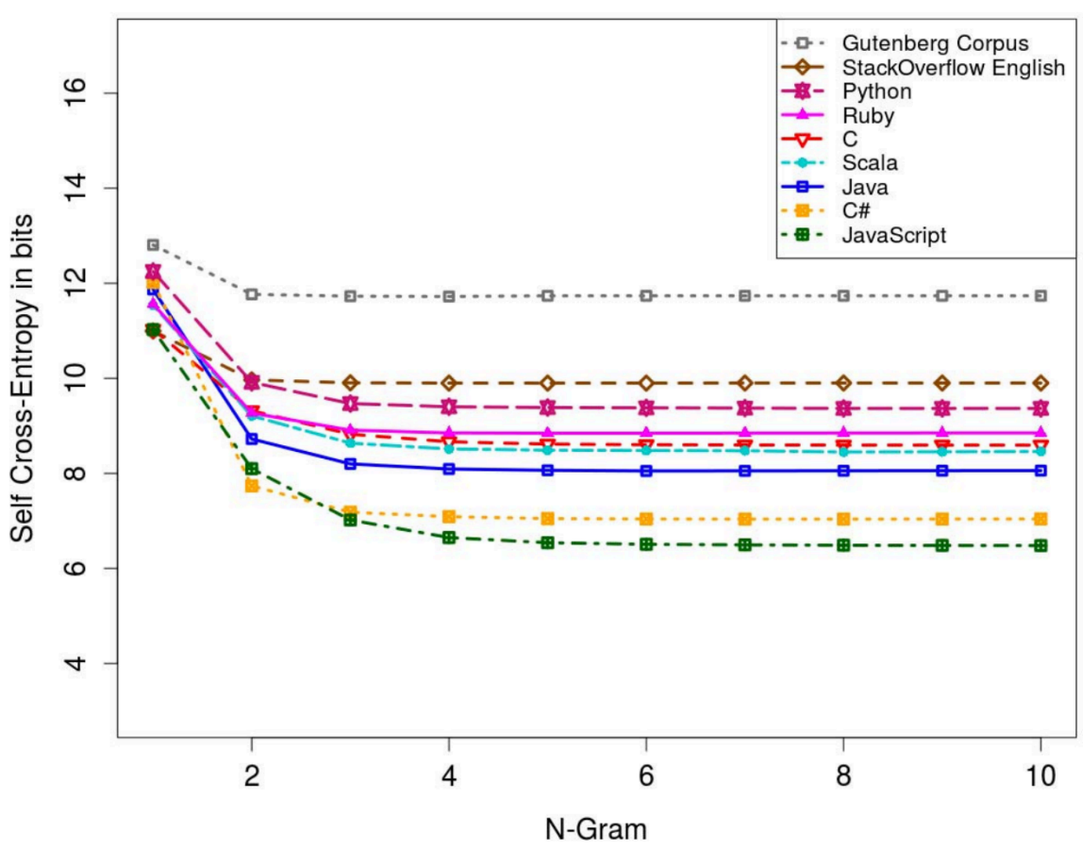

Cross−entropy
6.0
5.5
5.0
4.5
4.0
3.5
3.0
2.5
116
22
21
23
15
86
26
135
118
31
Self Cross−entropy
Cross−entropy of Alternative Ubuntu Categories
Admin
Doc
Graphics
Interpreters
Mail
Net
Sound
Tex
Text
Web
Corpus Categories

What about Stopwords?

- Rahman, M, Palani, D, & Rigby, P C (2019, May) Natural software revisited In IEEE/ACM 41st International Conference on Software Engineering (ICSE) (pp 37-48) IEEE
- 44% of all tokens are separators
- In NLP, punctuation is usually (but not necessarily!) removed



With separators



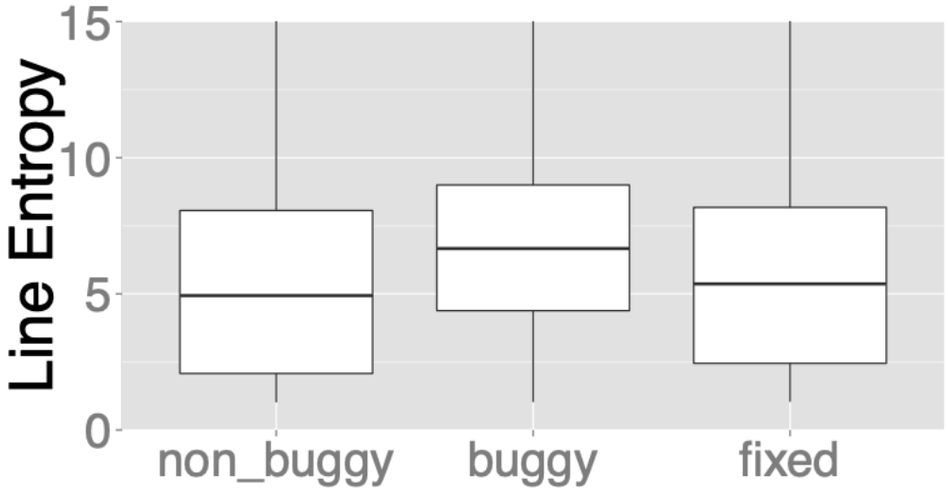
Without separators

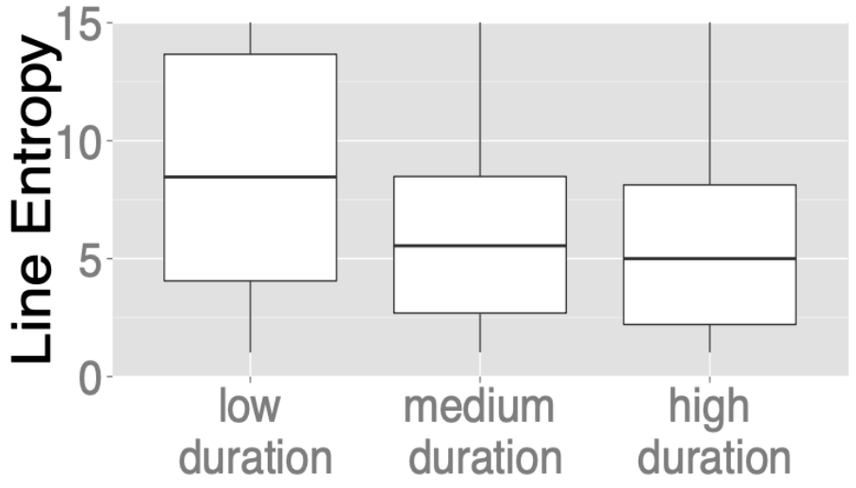
Is Buggy Code Less Natural?

- Ray, B, Hellendoorn, V, Godhane, S, Tu, Z, Bacchelli, A, & Devanbu, P (2016, May) On the "naturalness" of buggy code. In IEEE/ACM 38th International Conference on Software Engineering (ICSE) (pp 428-439) IEEE
- Used 7139 bug fix commits from 10 different Java projects to compare buggy with non-buggy code

Bug-fix threshold	**buggy vs. non-buggy (RQ1)** Entropy diff. (bug > non-bug)	Cohen's d effect	**buggy vs. fixed (RQ2)** Entropy diff. (bug > fix)	Cohen's d effect	%Unique bugs detected
1	1.95 to 2.00	0.61	1.58 to 1.67	0.60	14.92
2	1.68 to 1.72	0.53	1.43 to 1.51	0.52	23.93
4	1.37 to 1.40	0.43	1.16 to 1.23	0.41	34.65
7	1.15 to 1.17	0.36	0.98 to 1.04	0.34	43.94
15	0.91 to 0.93	0.29	0.75 to 0.81	0.26	55.91
20	0.81 to 0.83	0.25	0.62 to 0.68	0.21	60.16
50	0.58 to 0.59	0.18	0.39 to 0.44	0.13	71.68
100	0.55 to 0.57	0.17	0.36 to 0.41	0.12	80.96
Overall	0.86 to 0.87	0.26	0.69 to 0.74	0.19	100.00

Is Buggy Code Less Natural?





Is Buggy Code Less Natural?

Example 1 : Wrong Initialization Value
Facebook-Android-SDK (2012-11-20)
File: `Session.java`
Entropy dropped after bugfix : **4.12028**

```
  if (newState.isClosed()) {
    // Before (entropy = 6.07042):
-   this.tokenInfo = null;
    // After (entropy = 1.95014):
+   this.tokenInfo = AccessToken.createEmptyToken
                       (Collections.<String>emptyList());

  }
  ...
```

Is Buggy Code Less Natural?

Example 2 : Wrong Method Call
Netty (2013-08-20)
File: `ThreadPerChannelEventLoopGroup.java`

Entropy dropped after bugfix : **4.6257**

```
   if (isTerminated()) {
      // Before (entropy = 5.96485):
-     terminationFuture.setSuccess(null);
      // After (entropy = 1.33915):
+     terminationFuture.trySuccess(null);
```

Is Buggy Code Less Natural?

Example 3 : Unhandled Exception
Lucene (2002-03-15)
File: `FSDirectory.java`
Entropy dropped after bugfix : **3.87426**

```
  if (!directory.exists())
    // Before (entropy = 9.213675):
-   directory.mkdir();
    // After (entropy = 5.33941):
+   if (!directory.mkdir())
+     throw new IOException
            ("Cannot create directory:   " +
                directory);
```